CampusHub 校园互助平台 - 详细设计文档

项目名称： CampusHub 校园互助平台

设计阶段： Phase 3 详细设计文档（整合版）

技术架构： Vue3 + SpringBoot3 模块化单体架构

分层架构： Controller → Service → Repository → Entity

最后更新： 2026-05-17

一、 设计概述

本项目为 CampusHub 校园互助平台系统设计文档。本次详细设计全覆盖系统 P0/P1/P2 全部业务功能，按照后端标准四层架构拆分，从数据实体、设计模式、业务服务、接口控制四个维度完成全系统类结构梳理，完全满足校园跑腿、技能互助、二手交易、订单流转、实名认证、信用体系、即时聊天、后台管理全流程开发规范。

系统设计视角分为四大核心部分：

- **核心实体与数据库关系（图1）**：涵盖系统底层数据模型及各实体间的物理与逻辑映射。
- **系统设计模式实现（图2）**：通过引入设计模式解决复杂业务状态流转与扩展性痛点。
- **业务 Service 服务层结构（图3）**：定义核心业务逻辑、操作契约及跨模块交互规范。
- **前端请求 Controller 控制层（图4）**：确立符合 RESTful 风格的接口路径、入参与出参规范。

二、 核心实体关系设计

2.1 系统全局枚举类

1. 用户角色枚举 UserRole

枚举值	业务说明
STUDENT	普通在校学生用户
ADMIN	平台后台管理员

2. 用户账号状态 *UserStatus*

枚举值	业务说明
ACTIVE	账号正常可用
BANNED	账号违规封禁
PENDING_CERT	未完成实名认证

3. 实名认证类型 *CertType*

枚举值	业务说明
STUDENT_ID	学号线上核验认证
STUDENT_CARD	学生证图片人工审核

4. 认证审核状态 *CertStatus*

枚举值	业务说明
PENDING	等待管理员审核
APPROVED	实名认证通过
REJECTED	实名认证驳回

5. 需求板块分类 *DemandCategory*

枚举值	业务说明
ERRAND	校园跑腿代办
SKILL	电脑/软件技能求助
STUDY	学习资料、占座互助
SECONDHAND	校园二手物品交易

6. 需求发布状态 *DemandStatus*

枚举值	业务说明
OPEN	公开可接单状态
ACCEPTED	已被用户接单
IN_PROGRESS	服务进行中
COMPLETED	需求服务完结
CANCELLED	发布者主动取消
EXPIRED	加急需求自动过期

7. 报酬结算类型 *RewardType*

枚举值	业务说明
MONEY	现金现金报酬
POINT	平台积分报酬

8. 订单流转状态 *OrderStatus*

枚举值	业务说明
PENDING_ACCEPT	等待用户接单
ACCEPTED	成功接单
IN_PROGRESS	服务执行中
PENDING_CONFIRM	等待需求方确认完成
COMPLETED	订单正式完结
CANCELLED	订单取消终止

9. 评价身份角色 *ReviewerRole*

枚举值	业务说明
PUBLISHER	需求发起方评价
ACCEPTOR	接单服务方评价

10. 纠纷仲裁状态 *ArbitrationStatus*

枚举值	业务说明
PENDING	等待管理员处理
PROCESSING	仲裁处理中
RESOLVED	仲裁处理完毕
DISMISSED	仲裁申请驳回

11. 站内消息通知类型 *NotificationType*

枚举值	业务说明
ORDER_ACCEPTED	需求被接单通知
ORDER_COMPLETED	订单完成通知
ORDER_CANCELLED	订单取消通知
REVIEW_RECEIVED	收到用户评价
CREDIT_CHANGED	个人信用分变动
SYSTEM_ANNOUNCEMENT	平台系统公告
ARBITRATION_UPDATE	纠纷仲裁进度更新

12. 聊天消息类型 *MessageType*

枚举值	业务说明
TEXT	普通文字消息
IMAGE	图片聊天消息
SYSTEM	会话系统提示消息

2.2 核心业务实体类完整定义

1. *User* 用户实体

核心属性：

属性名	数据类型	字段说明
id	Long	主键自增 ID
username	String	唯一登录账号
passwordHash	String	BCrypt 加密密码
phone	String	加密存储手机号
email	String	预留邮箱账号
studentId	String	唯一在校学号
realName	String	用户真实姓名
avatarUrl	String	头像文件地址
creditScore	Integer	个人信用分(初始100)
role	UserRole	用户身份角色
status	UserStatus	账号使用状态
createdAt	LocalDateTime	账号注册时间
updatedAt	LocalDateTime	资料最后修改时间

内置业务方法：

- `isCreditSufficient()`: 判断信用分是否≥60，限制低信用用户接单。
- `addCredit(int amount)`: 增加用户信用分值。
- `deductCredit(int amount)`: 扣除用户信用分值。

2. Certification 实名认证实体

核心属性：

属性名	数据类型	字段说明
id	Long	主键 ID
userId	Long	关联用户 ID
certType	CertType	认证方式
studentId	String	核验学号
cardImageUrl	String	学生证凭证图片
status	CertStatus	审核状态
rejectReason	String	审核驳回原因
reviewedBy	Long	审核管理员 ID
reviewedAt	LocalDateTime	审核操作时间
createdAt	LocalDateTime	提交认证时间

内置方法：

- `approve(Long reviewerId)`: 审核通过认证。
- `reject(Long reviewerId, String reason)`: 驳回实名认证申请。

3. CreditRecord 信用分变动记录

核心属性：

属性名	数据类型	字段说明
id	Long	主键
userId	Long	所属用户 ID
changeAmount	Integer	信用变动数值
reason	String	分值变动原因
relatedOrderId	Long	关联业务订单 ID
createdAt	LocalDateTime	变动记录时间

4. Demand 校园需求发布实体

核心属性：

属性名	数据类型	字段说明
id	Long	主键 ID
publisherId	Long	需求发布人 ID
title	String	需求简短标题
description	String	需求详细描述
category	DemandCategory	需求业务板块
rewardAmount	BigDecimal	服务报酬金额
rewardType	RewardType	报酬结算形式
status	DemandStatus	需求整体状态
isUrgent	Boolean	是否加急需求
urgentDeadline	LocalDateTime	加急截止时效
locationTag	String	校园位置标签
createdAt	LocalDateTime	需求发布时间
updatedAt	LocalDateTime	需求编辑时间

内置方法：

- `isOpen()`: 判断需求是否可接单。
- `markAsAccepted()`: 标记需求已接单。
- `markAsCompleted()`: 标记需求服务完成。
- `cancel()`: 发布者取消需求订单。

5. DemandImage 需求配图实体

存储需求发布附带多张实拍图片，关联需求 ID 实现一对多绑定。

6. Tag 校园位置标签实体

实现需求与校园地点多对多关联，用于就近匹配接单用户。

7. Order 业务订单核心实体

核心属性：

属性名	数据类型	字段说明
id	Long	订单主键
demandId	Long	绑定对应需求 ID
publisherId	Long	需求发起方 ID
acceptorId	Long	接单服务方 ID
status	OrderStatus	订单流转状态
acceptedAt	LocalDateTime	接单成功时间
completedAt	LocalDateTime	订单完成时间
cancelledAt	LocalDateTime	订单取消时间
cancelReason	String	订单取消原因
autoConfirmDeadline	LocalDateTime	24小时自动确认时效
createdAt	LocalDateTime	订单创建时间
updatedAt	LocalDateTime	订单状态更新时间

内置方法：

- `canAccept()`: 校验当前订单是否允许接单。
- `canCancel()`: 校验当前订单是否允许取消。
- `isAutoConfirmExpired()`: 判断是否触发自动确认完成。

8. OrderStatusLog 订单状态变更日志

完整记录订单每一次状态流转、操作人、操作备注，实现业务全链路溯源。

9. Review 双向评价实体

实现需求方与服务方双向星级评价，低分评价（≤3星）自动触发纠纷仲裁。

10. ArbitrationTicket 纠纷仲裁工单

针对恶意差评、服务违约创建后台仲裁工单，由管理员统一处理判定。

11. Notification 站内统一通知

全平台业务消息统一推送载体，支持已读/未读、批量已读、未读统计。

12. Conversation 聊天会话实体

搭建用户一对一私聊会话，绑定需求实现业务专属聊天窗口。

13. Message 聊天消息实体

存储会话内所有文字、图片聊天记录，支持消息已读状态标记。

14. AdminOperationLog 管理员操作日志

记录后台所有封禁用户、审核认证、处理仲裁等管理行为，便于后台风控审计。

2.3 实体类关系汇总表

主体类	关联关系	关联类	数量基数	业务含义
User	一对一	Certification	1 : 0..1	一个用户仅有一条实名认证记录
User	一对多	CreditRecord	1 : N	用户多条信用变动记录
User	一对多	Demand	1 : N	用户可发布多条校园需求
User	一对多	Order	1 : N	用户可作为发布方或接单方持有订单
Demand	一对一	Order	1 : 0..1	一条需求仅生成一笔正式订单
Demand	一对多	DemandImage	1 : N	需求可上传多张配图
Demand	多对多	Tag	N : M	需求绑定多个校园位置/技能标签
Order	一对多	OrderStatusLog	1 : N	订单多条状态变更日志
Order	一对多	Review	1 : 0..2	订单双方各可提交一条评价
Conversation	一对多	Message	1 : N	会话承载多条聊天消息
Review	一对一	ArbitrationTicket	1 : 0..1	低分评价可自动/手动发起一次仲裁

三、 系统核心设计模式类图实现

3.1 策略模式：订单状态流转管控

> **业务痛点：**订单存在 6 种流转状态（待接单、已接单、进行中、待确认、已完成、已取消），不同状态下用户可执行的操作严格受限。若使用原生 `if-else` 或 `switch` 链硬编码进行判断，代码结构将极其臃肿，且未来新增状态（如“争议中”）时扩展性极差。

解决方案：定义统一订单状态策略接口，每种状态独立实现对应业务权限，通过上下文动态切换策略。

- **顶层策略接口 IOrderStateStrategy**：提供核心校验方法如 `canAccept`、`canComplete`、`canCancel`、`canConfirm` 并在接口内声明获取合法状态流转列表。
- **上下文管理类 OrderStateContext**：自动根据订单当前状态绑定对应策略类，统一对外提供状态权限判断。
- **六大状态策略实现类**：包括待接单策略、已接单策略、进行中策略、待确认策略、已完成策略、已取消策略，各自精细定义专属的操作行为与流转权限。

3.2 工厂模式：多类型需求创建校验

业务痛点：校园跑腿（ERRAND）、技能互助（SKILL）、学习占座（STUDY）、二手物品（SECONDHAND）四类不同版块的需求在创建时校验规则各不相同。例如，跑腿必须有位置标签，二手物品必须有配图。硬编码在业务方法中会造成严重的逻辑耦合。

解决方案：搭建需求工厂类，绑定不同板块专属校验器，发布需求自动匹配规则完成参数校验。

- **校验顶层接口 DemandValidator**：定义统一的校验契约方法 `validate(DemandParam param)`。
- **四大业务板块独立校验实现类：**ErrandValidator（强制位置）、SkillValidator（强制技能标签）、StudyValidator（强制时间段）、SecondhandValidator（强制包含实物配图）。
- **需求工厂 DemandFactory**：统一接收发布参数，根据 `category` 从动态注册的 Map 容器中匹配对应的校验规则，合法后生成正式的需求实体。

3.3 观察者模式：业务事件解耦通知

业务痛点：当订单发生变更（接单、完成、取消）时，系统需要同时执行多项附加操作，如推送站内消息、增减用户信用分值、更新需求整体状态等。若串行编码在服务层中，将引发严重的业务耦合与事务延长问题。

解决方案：基于 Spring 事件发布订阅机制实现业务事件完全解耦。业务核心层仅负责发布相应事件，多个独立的监听类自主处理后续衍生逻辑。

- **订单事件统一发布器**：负责包装订单变更状态并推送事件。
- **消息通知监听器**：异步接收并处理事件，自动封装并向目标用户推送特定的站内通知。
- **信用分值监听器**：根据订单完结或违约事件，调用信用服务进行自动加减分操作。

3.4 组合策略模式：智能用户匹配推荐

为发布的校园互助需求智能推荐合适的接单用户，支持地理位置、技能标签、信用分值等多维度综合排序。

- **统一匹配顶层接口 DemandMatcher**：定义用户匹配推荐算法标准。
- **单维度精准匹配器**：位置匹配器（LocationMatcher）、技能匹配器（SkillMatcher）、信用分匹配器（CreditScoreMatcher）。
- **组合综合匹配器**：对上述单维度匹配结果进行加权复合排序，输出最优的智能推荐接单用户列表。

四、 SOLID 原则审查与修改清单

团队对照面向对象设计的 SOLID 原则，逐条审查了最初由 AI 第一版生成的原始类图，记录了其中违反开发规范与架构设计原则的 7 处严重设计缺陷，并制定并实施了最终的修正方案。

4.1 SOLID 审查明细

原则	AI是否违反	违反说明	修正方案
S - 单一职责原则 (SRP)	是 (2处)	1. `OrderService`：同时负责接单、确认、取消、计算信用分变动、发送通知（5项职责）。信用分、通知分别属于用户信用与消息子域，不应高度耦合。 2. `DemandService`：既负责需求CRUD，又内嵌处理标签（Tag）的全面增删改查。	1. 拆分职责：将信用逻辑提取至 `CreditService`，通知发送提取至状态和业务编排。 2. 拆分标签管理：独立抽出 `TagService`，需求服务仅在创建时通
O - 开闭原则 (OCP)	是 (2处)	1. 需求创建：`DemandService.createDemand()` 内部通过 `switch(category)` 硬编码判断需求类型并做差异校验，新增板块需改动主干分支。 2. 订单流转：状态流转操作在 `OrderService` 中通过庞大的 `if-else` 链条判断 `OrderStatus`。	1. 引入工厂模式：定义 `DemandValidator` 接口，并为跑腿、技能 Map 映射调度。 2. 引入策略模式：定义 `IOrderStateStrategy` 接口，为6种状态
L - 里氏替换原则 (LSP)	是 (1处)	AI 设计了 `UrgentDemand extends Demand`，重写 `getExpiryTime()` 返回加急截止时间。导致 `DemandService.listDemands()` 统一按过期时间排序时行为不一致，破坏了父类方法契约。	**用组合替代继承：** 1. 彻底移除 `UrgentDemand` 子类。 2. 在 `Demand` 实体中新增 `isUrgent` (Boolean) 和 `urgentDeadlin` 3. Service 层通过自定义 `Comparator` 统一处理排序。
I - 接口隔离原则 (ISP)	是 (1处)	AI 原始设计定义了一个臃肿的 `ICampusService` 大接口，整合了用户、需求、订单、评价等所有方法。导致 `UserServiceImpl` 被迫实现无关的需求与评价方法，只能空实现。	**拆分细粒度接口：** 解耦拆分为独立的 `UserService`、`DemandService`、`OrderSer` `NotificationService` 等细粒度专业接口。
D - 依赖倒转原则 (DIP)	是 (1处)	1. `OrderService` 通过 `new NotificationServiceImpl()` 直接创建硬编码依赖，未使用控制反转容器。 2. 跨模块场景下，`OrderService` 直接注入了 `DemandRepository`，违反跨模块访问开发规范。	1. 改用 Spring 容器的依赖注入机制，通过 `@Autowired` 注入接口。 2. 跨模块调用强制改为注入并调用 `DemandService` 的公开方法，

4.2 违规与修正统计

统计维度	单一职责 (SRP)	开闭原则 (OCP)	里氏替换 (LSP)	接口隔离 (ISP)	依赖倒转 (DIP)	总计
AI 原始设计违规数	2 处	2 处	1 处	1 处	1 处	**7 处**
团队已完全修正数	2 处	2 处	1 处	1 处	1 处	**7 处**

五、 架构组件结构设计

5.1 Service 业务服务层结构

- **认证权限模块 (auth)**:
  - `AuthService`：处理用户注册、登录验证、登出注销、Token 安全校验。
  - `JwtTokenProvider`：负责 JWT 核心令牌的生成、解析、过期时效核验。
- **用户中心模块 (user)**:
  - `UserService`：提供个人资料信息查询、基本资料修改、公开敏感信息脱敏查询。
  - `CertificationService`：实名认证申请提交、管理员后台审核、用户认证状态追踪。
  - `CreditService`：管理用户信用积分扣减、接单权限门槛校验、信用记录流水追踪。
- **核心业务模块 (experiment)**:
  - `DemandService`：实现互助需求发布、内容编辑、主动取消、分页高级检索与关键词模糊搜索。
  - `OrderService`：控制用户接单流转、标记送达、确认验收、申请取消及 24 小时定时自动确认机制。
  - `ReviewService`：支持完结订单双向评价提交、用户历史评级列表调取。
  - `ArbitrationService`：负责系统低分触发或申诉仲裁工单创建、后台工单处理及待办审计管理。
- **消息通知模块 (notification)**:
  - `NotificationService`：负责全平台异步业务消息创建推送、未读计数统计、单条/全部已读状态打标。
- **即时通讯模块 (chat)**:
  - `ConversationService`：一对一聊天会话生命周期创建、聊天消息实时发送、历史通讯记录轮询调取、会话已读同步。
- **智能推荐模块 (recommend)**:
  - `RecommendationService`：利用组合策略输出用户偏好需求列表、接单用户智能就近匹配。
- **后台管理模块 (admin)**:
  - `AdminService`：提供平台核心数据统计看板、违规用户封禁与解封控制、后台全局列表审计。

5.2 Controller 接口控制层规范

所有控制层接口统一遵循以下通用约定：

- 统一返回全局响应实体结构：`ApiResponse<code, message, data>`。
- 所有鉴权接口统一从 HTTP 请求头中提取凭证：`Authorization: Bearer <JWT_TOKEN>`。
- 接口控制层划分与路由根路径定义：
  - **认证接口**：`AuthController` → `/api/auth`
  - **实名认证接口**：`CertificationController` → `/api/certifications`
  - **用户信息接口**：`UserController` → `/api/users`
  - **需求发布接口**：`DemandController` → `/api/demands`
  - **订单业务接口**：`OrderController` → `/api/orders`
  - **评价管理接口**：`ReviewController` → `/api/reviews`
  - **纠纷仲裁接口**：`ArbitrationController` → `/api/arbitrations`
  - **站内通知接口**：`NotificationController` → `/api/notifications`
  - **聊天会话接口**：`ConversationController` → `/api/conversations`
  - **后台管理接口**：`AdminController` → `/api/admin`

六、 API 规范文档

6.1 通用约定

项目	说明
通信协议	HTTPS (生产环境) / HTTP (本地开发环境)
服务域名	生产: `https://campushub.example.com`; 本地: `http://localhost:8080`
内容类型 (Content-Type)	`application/json; charset=utf-8`
字符编码	UTF-8
时间格式标准	所有时间字段统一使用 ISO8601 格式，例如：`2026-05-17T14:30:00Z`
命名空间规范	URL 路径采用小写、复数名词、短横线分隔（例：`/api/demands`）；请求体与响应体字段统一使用小驼峰格式（camelCase，例：`publisherId`）。

统一基础响应 JSON 结构：

```
{
    "code": 20000,
    "message": "success",
```

```
    "data": {}
}
```

字段	类型	说明
code	Integer	业务状态码（具体参见错误码映射表）
message	String	提示信息说明
data	Object / null	实际承载的返回业务数据，在操作失败时该字段固定为 null

统一分页请求参数规范：

参数	类型	默认值	说明
page	Integer	0	页码（基于 0 开始计数）
size	Integer	20	每页拉取的记录条数

统一分页响应 JSON 结构：

```
{
    "code": 20000,
    "message": "success",
    "data": {
        "content": [],
        "page": 0,
        "size": 20,
        "totalElements": 100,
        "totalPages": 5
    }
}
```

6.2 错误码定义表

错误码范围	类别	说明
20000	成功	操作成功
40000-40099	参数错误	请求体关键参数缺失、类型不匹配或格式校验不合法
40100-40199	认证错误	用户未登录、安全 Token 缺失、已过期或签名无效
40300-40399	授权错误	当前身份权限不足、用户信用积分过低、未实名认证或账号被封禁
40400-40499	资源不存在	请求的目标用户、互助需求、对应订单或实体不存在
40900-40999	业务冲突	当前业务状态不允许此项操作、重复提交或数据冲突
50000-50099	服务端错误	服务器内部异常或数据库异常

具体核心业务错误码映射表：

具体错误码	对应业务说明
20000	操作成功
40001	参数缺失或格式错误
40002	参数值不在允许范围内
40100	未提供认证 Token
40101	Token 已过期
40102	Token 无效
40300	权限不足
40301	信用分不足(<60), 无法接单
40302	未完成实名认证，操作拦截
40303	账号已被违规封禁
40400	资源不存在
40401	用户不存在
40402	需求不存在
40403	订单不存在
40900	业务当前状态不允许此操作
40901	需求已被他人提前接单
40902	订单已取消，不可执行此操作
40903	已提交过评价，不可重复评价
40904	用户名已被占用
40905	该学号已被注册
50000	服务器内部错误

6.3 核心接口定义

1. 用户注册

- **接口路径与方法：** `POST /api/auth/register`

- **认证要求：** 无需认证
- **请求体参数：**

参数	类型	必填	说明
username	String	是	登录名，4-20位字母/数字/下划线
password	String	是	密码，6-32位明文串
studentId	String	是	高校唯一在校学号
realName	String	...	真实姓名
phone	String	否	手机号
email	String	否	邮箱

- **成功响应示例：**

```
{
    "code": 20000,
    "message": "注册成功",
    "data": {
        "id": 1,
        "username": "alice",
        "studentId": "2024001001",
        "realName": "张三",
        "role": "STUDENT",
        "status": "PENDING_CERT",
        "creditScore": 100,
        "createdAt": "2026-05-17T10:00:00Z"
    }
}
```

2. 用户登录

- **接口路径与方法：** `POST /api/auth/login`
- **认证要求：** 无需认证
- **请求体参数：**

参数	类型	必填	说明
username	String	是	唯一登录名
password	String	是	登录密码

- **成功响应示例：**

```
{
    "code": 20000,
    "message": "登录成功",
    "data": {
        "token": "eyJhbGciOiJIUzI1NiIs...",
        "expiresAt": "2026-05-18T10:00:00Z",
        "user": {
            "id": 1,
            "username": "alice",
            "role": "STUDENT",
            "status": "ACTIVE",
            "creditScore": 100
        }
    }
}
```

3. 发布需求

- **接口路径与方法：** `POST /api/demands`
- **认证要求：** 需提供合法用户 Token，且必须完成实名认证
- **请求体参数说明：**

参数	类型	必填	说明
title	String	是	需求标题，2-50字
description	String	...	详细描述，10-500字
category	String	是	板块：ERRAND / SKILL / STUDY / SECONDHAND
rewardAmount	BigDecimal	是	报酬金额，必须 ≥ 0
rewardType	String	是	结算形式：MONEY / POINT
isUrgent	Boolean	否	是否加急，默认为 false
urgentDeadline	String	否	加急截止时间（isUrgent 为 true 时必填）
locationTag	String	是	位置（如"南区快递点"、"一教"、"图书馆"）
tagIds	Long[]	否	关联标签 ID 列表
imageUrls	String[]	否	需求图片附带的 URL 列表

- **板块额外工厂校验规则：**
  - `ERRAND`（校园跑腿）：必须显式填写 `locationTag`。
  - `SKILL`（技能求助）：必须至少关联 1 个技能标签 `tagIds`。
  - `STUDY`（学习互助）：必须填写具体互助时间段信息。
  - `SECONDHAND`（二手物品）：必须在 `imageUrls` 中上传至少 1 张实物外观图片。

4. 浏览需求列表（含高级筛选）

- **接口路径与方法：** `GET /api/demands`
- **认证要求：** 需认证
- **查询 URL 参数：** 包含 `category`、`status`、`keyword`（模糊搜索）、`locationTag`、`isUrgent`、`sortBy`（支持按发布时间、报酬金额或加急截止时间排序）、`sortDir`（asc/desc）以及基础分页参数。

5. 接单操作

- **接口路径与方法：** `POST /api/orders`
- **认证要求：** 需完成实名认证且个人实时信用积分必须 ≥ 60 分。
- **请求体参数：**

参数	类型	必填	说明
demandId	Long	是	打算承接的目标校园需求主键 ID

- **失败拦截响应：** 若需求已被承接，返回 `40901`；若信用分不足，拦截并返回状态码 `40301`（提示“信用分不足，无法接单”）。

6. 取消订单（带有信用扣分机制）

- **接口路径与方法：** `PUT /api/orders/{id}/cancel`
- **业务机制：** 成功承接订单后 10 分钟内取消，系统判定为无责取消，信用积分不受惩罚；若超出 10 分钟宽限期实施超时违约取消，将触发观察者事件，自动扣除接单服务方 10 分个人信用分。

7. 提交双向评价与差评举证

- **接口路径与方法：** `POST /api/reviews`
- **参数要求：**

参数	类型	必填	说明
orderId	Long	是	关联业务订单 ID
rating	Integer	是	评分星级，范围 1 - 5 星
content	String	条件	当评分 ≤ 3 星时，必须填写不少于 15 字的差评原因
imageUrls	String[]	条件	当评分 ≤ 3 星时，必须至少上传 1 张截图作为证据链

- **业务流转：** 评分 ≤ 3 星的低分差评提交后，系统将通过业务编排自动触发并向后台创建对应的 `ArbitrationTicket` 纠纷仲裁工单，交由管理员进行介入处理。

七、 数据库设计

7.1 物理实体与主外键设计说明

数据表	主键 (PK)	关键外键 (FK) / 约束	业务设计与范式说明
users	id	username (UNIQUE) student_id (UNIQUE)	用户账户主表，记录真实姓名、加密凭证及最新信用积分。
certifications	id	user_id → users.id (UNIQUE)	维护一对一实名认证状态，防止单用户存在多条待审核申请。
credit_records	id	user_id → users.id	信用分增减变动历史流水表，追溯变动关联订单。
demands	id	publisher_id → users.id	存储校园需求核心内容、涉及版块、报酬类型及发布状态。
demand_images	id	demand_id → demands.id	维护需求附带配图，采用 sort_order 实现内部精确排序。
tags	id	name (UNIQUE)	全局公共标签字典库（位置、技能标签）。
demand_tags	(demand_id, tag_id)	联合主键，分别引用两张表	实现需求与标签之间的典型多对多（N:M）关系物理表。
orders	id	demand_id → demands.id (UNIQUE)	一个有效需求最多只能衍生出一笔接单订单，约束互斥。
order_status_logs	id	order_id → orders.id	全量记录订单每一次状态变更流转明细，以便全链路溯源。
reviews	id	order_id, reviewer_id (联合唯一)	双向星级评价表，一笔订单中的一方参与者至多限评一次。
arbitration_tickets	id	order_id → orders.id	针对违约或差评投诉自动/手动生成的管理员仲裁申诉工单。
notifications	id	user_id → users.id	站内个人通知主表。
conversations	id	联合唯一索引约束双方 ID	双方沟通的聊天会话实体，可选择性绑定特定需求主键。

数据表	主键 (PK)	关键外键 (FK) / 约束	业务设计与范式说明
messages	id	conversation_id → conversations.id	会话内的具体聊天消息明细存储表。
admin_operation_logs	id	admin_id → users.id	管理员实施封禁、审批、处理仲裁的审计日志。

7.2 完整建表 SQL 脚本 (DDL)

以下为平台通过第三范式建模的完整物理建表语句，兼容标准 PostgreSQL 规范：

```
CREATE TYPE user_role AS ENUM ('STUDENT', 'ADMIN');
CREATE TYPE user_status AS ENUM ('ACTIVE', 'BANNED', 'PENDING_CERT');
CREATE TYPE cert_type AS ENUM ('STUDENT_ID', 'STUDENT_CARD');
CREATE TYPE cert_status AS ENUM ('PENDING', 'APPROVED', 'REJECTED');
CREATE TYPE demand_category AS ENUM ('ERRAND', 'SKILL', 'STUDY', 'SECONDHAND');
CREATE TYPE demand_status AS ENUM ('OPEN', 'ACCEPTED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED',
'EXPIRED');
CREATE TYPE reward_type AS ENUM ('MONEY', 'POINT');
CREATE TYPE order_status AS ENUM ('PENDING_ACCEPT', 'ACCEPTED', 'IN_PROGRESS',
'PENDING_CONFIRM', 'COMPLETED', 'CANCELLED');
CREATE TYPE reviewer_role AS ENUM ('PUBLISHER', 'ACCEPTOR');
CREATE TYPE arbitration_status AS ENUM ('PENDING', 'PROCESSING', 'RESOLVED', 'DISMISSED');
CREATE TYPE notification_type AS ENUM (
    'ORDER_ACCEPTED', 'ORDER_COMPLETED', 'ORDER_CANCELLED',
    'REVIEW_RECEIVED', 'CREDIT_CHANGED', 'SYSTEM_ANNOUNCEMENT', 'ARBITRATION_UPDATE'
);
CREATE TYPE message_type AS ENUM ('TEXT', 'IMAGE', 'SYSTEM');

CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(200) NOT NULL,
    phone_cipher TEXT,
    phone_hash VARCHAR(128),
    email VARCHAR(200),
    student_id VARCHAR(50) NOT NULL UNIQUE,
    real_name VARCHAR(200) NOT NULL,
    avatar_url TEXT,
    credit_score INT NOT NULL DEFAULT 100 CHECK (credit_score >= 0),
    role user_role NOT NULL DEFAULT 'STUDENT',
    status user_status NOT NULL DEFAULT 'PENDING_CERT',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE certifications (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,
    cert_type cert_type NOT NULL,
    student_id VARCHAR(50) NOT NULL,
    card_image_url TEXT,
    status cert_status NOT NULL DEFAULT 'PENDING',
    reject_reason TEXT,
    reviewed_by BIGINT REFERENCES users(id) ON DELETE SET NULL,
    reviewed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_cert_card_image CHECK (cert_type <> 'STUDENT_CARD' OR card_image_url IS NOT
NULL)
```

```
);

CREATE TABLE credit_records (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    change_amount INT NOT NULL,
    reason VARCHAR(255) NOT NULL,
    related_order_id BIGINT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE demands (
    id BIGSERIAL PRIMARY KEY,
    publisher_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    category demand_category NOT NULL,
    reward_amount NUMERIC(12, 2) NOT NULL DEFAULT 0 CHECK (reward_amount >= 0),
    reward_type reward_type NOT NULL DEFAULT 'MONEY',
    status demand_status NOT NULL DEFAULT 'OPEN',
    is_urgent BOOLEAN NOT NULL DEFAULT false,
    urgent_deadline TIMESTAMPTZ,
    location_tag VARCHAR(200) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_urgent_deadline CHECK (is_urgent = false OR urgent_deadline IS NOT NULL)
);

CREATE TABLE demand_images (
    id BIGSERIAL PRIMARY KEY,
    demand_id BIGINT NOT NULL REFERENCES demands(id) ON DELETE CASCADE,
    image_url TEXT NOT NULL,
    sort_order INT NOT NULL DEFAULT 0,
    UNIQUE (demand_id, sort_order)
);

CREATE TABLE tags (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE demand_tags (
    demand_id BIGINT NOT NULL REFERENCES demands(id) ON DELETE CASCADE,
    tag_id BIGINT NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
    PRIMARY KEY (demand_id, tag_id)
);

CREATE TABLE orders (
    id BIGSERIAL PRIMARY KEY,
    demand_id BIGINT NOT NULL UNIQUE REFERENCES demands(id) ON DELETE RESTRICT,
    publisher_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    acceptor_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    status order_status NOT NULL DEFAULT 'ACCEPTED',
    accepted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ,
    cancelled_at TIMESTAMPTZ,
    cancel_reason TEXT,
    auto_confirm_deadline TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_order_not_self_accept CHECK (publisher_id <> acceptor_id)
);

ALTER TABLE credit_records
    ADD CONSTRAINT fk_credit_record_order
```

```
    FOREIGN KEY (related_order_id) REFERENCES orders(id) ON DELETE SET NULL;

CREATE TABLE order_status_logs (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    from_status order_status,
    to_status order_status NOT NULL,
    operator_id BIGINT REFERENCES users(id) ON DELETE SET NULL,
    remark TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE reviews (
    id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    reviewer_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    reviewee_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    reviewer_role reviewer_role NOT NULL,
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    content TEXT,
    image_urls JSONB NOT NULL DEFAULT '[]'::jsonb,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (order_id, reviewer_id),
    UNIQUE (order_id, reviewer_role),
    CONSTRAINT chk_review_not_self CHECK (reviewer_id <> reviewee_id),
    CONSTRAINT chk_low_rating_evidence CHECK (
        rating > 3 OR (content IS NOT NULL AND length(content) >= 15 AND
jsonb_array_length(image_urls) >= 1)
    )
);

CREATE TABLE arbitration_tickets (
    id BIGSERIAL PRIMARY KEY,
    review_id BIGINT REFERENCES reviews(id) ON DELETE SET NULL,
    order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    reporter_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    reason TEXT NOT NULL,
    evidence_urls JSONB NOT NULL DEFAULT '[]'::jsonb,
    status arbitration_status NOT NULL DEFAULT 'PENDING',
    resolution TEXT,
    resolved_by BIGINT REFERENCES users(id) ON DELETE SET NULL,
    resolved_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE notifications (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type notification_type NOT NULL,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    related_id BIGINT,
    is_read BOOLEAN NOT NULL DEFAULT false,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE conversations (
    id BIGSERIAL PRIMARY KEY,
    user1_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    user2_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    related_demand_id BIGINT REFERENCES demands(id) ON DELETE SET NULL,
    last_message_content VARCHAR(1000),
    last_message_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_conversation_distinct_users CHECK (user1_id <> user2_id)
```

```
);

CREATE UNIQUE INDEX uq_conversation_pair_demand
    ON conversations (
        LEAST(user1_id, user2_id),
        GREATEST(user1_id, user2_id),
        COALESCE(related_demand_id, 0)
    );

CREATE TABLE messages (
    id BIGSERIAL PRIMARY KEY,
    conversation_id BIGINT NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    sender_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    content TEXT NOT NULL,
    message_type message_type NOT NULL DEFAULT 'TEXT',
    is_read BOOLEAN NOT NULL DEFAULT false,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE admin_operation_logs (
    id BIGSERIAL PRIMARY KEY,
    admin_id BIGINT NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    operation VARCHAR(200) NOT NULL,
    target_type VARCHAR(100) NOT NULL,
    target_id BIGINT,
    detail TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

7.3 高频查询索引设计说明

为确保系统在大规模并发及高频数据吞吐下的流畅度，对数据库底层表结构定制了高性能索引方案：

- **idx_users_phone_hash 与 idx_users_status**：使用手机号带盐不可逆哈希值构建 B-Tree 索引，支持应用层不暴露明文的精确高效去重与状态过滤。
- **idx_demands_status_created / category_status / location_status**：针对用户浏览大厅高频条件组合（按状态、版块、地点、发布时间）设计复合索引，完美规避全表扫描。
- **idx_demands_keyword (GIN 倒排索引)**：对需求标题与详情文本进行模糊合并分词，提供高效的站内关键词全文模糊检索支持。
- **idx_orders_publisher_status 与 idx_orders_acceptor_status**：为个人中心“我的发布”和“我的接单”列表提供极速状态拉取及倒排排序。
- **idx_messages_conversation_created**：针对私聊会话中根据聊天时间线倒序上拉历史消息记录设计，保证即时通讯的性能响应。

7.4 隐私安全与第三范式合规性

1. **数据隐私加固：** 用户敏感密码绝不存储明文，在应用层统一通过 BCrypt 自适应高强度加盐算法哈希后落库；手机号采用字段级对称加密，phone_cipher 存密文供页面解密脱敏展示，phone_hash 存储防重不可逆哈希。
2. **规范化建模：** 物理设计严格契合关系数据库第三范式（3NF）核心准则，用户、需求、订单、双向评价、系统通知等均完全原子化独立成表。多对多关联通过专门的关系交叉解耦表（如 demand_tags）拆解为两个 1:N 关系；状态变更有独立日志追踪，杜绝出现历史变动冗余字段带来的更新异常风险。